

Sincronizzazione nei Sistemi Distribuiti

Clock fisici, clock logici e ordine causale

Architetture dei Sistemi Distribuiti

Lezione 21

Un sistema distribuito non ha un osservatore globale immediato.

La domanda progettuale non è soltanto:

che ora era?

La domanda corretta è:

quale relazione temporale posso promettere all'interfaccia?

- 1 modello di eventi distribuiti;
- 2 clock fisico locale;
- 3 skew, drift, accuratezza e risoluzione;
- 4 wall-clock, monotonic clock, NTP e PTP;
- 5 intervalli di incertezza, timeout e lease;
- 6 happened-before;
- 7 clock di Lamport;
- 8 ordine totale deterministico;
- 9 vector clock;
- 10 causal delivery.

Consideriamo processi:

$$P = \{p_1, p_2, \dots, p_n\}$$

Ogni processo produce una sequenza locale di eventi:

$$E_i = \text{eventi prodotti da } p_i$$

Gli eventi includono:

- computazioni locali;
- invii e ricezioni di messaggi;
- letture e scritture di stato;
- operazioni applicative, per esempio SET $x \leftarrow 1$.

Dentro un processo l'ordine è noto:

`a precede localmente b`

Tra processi diversi l'ordine non è direttamente osservabile.

Cause:

- latenza variabile;
- messaggi persi, ritardati o riordinati;
- clock fisici non perfettamente allineati;
- eventi concorrenti;
- assenza di stato globale atomico.

Scenario: Log Apparentemente Impossibile

A 10:00:00.200 send m to B

B 10:00:00.100 receive m from A

Relazione reale:

`send(A,m) -> receive(B,m)`

Il log fisico locale sembra dire il contrario.

Conclusione:

un timestamp fisico locale non è una prova di ordine globale

Ogni nodo usa un proprio orologio fisico.

È utile per:

- log leggibili da operatori;
- audit e tracciamento;
- metriche operative;
- scadenze applicative;
- correlazione approssimata tra eventi.

Non basta per dimostrare causalità tra nodi diversi.

Clock skew

- differenza tra due clock nello stesso istante reale;
- esempio: A è avanti di 100 ms rispetto a B.

Clock drift

- divergenza progressiva tra clock;
- due oscillatori non avanzano esattamente alla stessa velocità;
- anche dopo una sincronizzazione i clock tornano a divergere.

Esempio: Skew

real time: 12:00:00.000

clock_A: 12:00:00.120

clock_B: 11:59:59.970

$\text{skew}(A,B) = 150 \text{ ms}$

Se A e B scrivono log con questi clock, l'ordinamento testuale dei timestamp può essere diverso dall'ordinamento reale.

- **Risoluzione:** il più piccolo incremento osservabile.
- **Precisione:** stabilità e ripetibilità delle misure.
- **Accuratezza:** vicinanza al tempo reale di riferimento.

Un clock può stampare nanosecondi ma essere avanti di due secondi.

Alta risoluzione non implica alta accuratezza.

Wall-clock

- rappresenta data e ora civile;
- può saltare avanti o indietro;
- serve per log, audit e interazione con operatori.

Monotonic clock

- non torna indietro;
- serve per durate, timeout e misure di latenza;
- non è necessariamente confrontabile tra nodi diversi.

`wall_clock()` $\rightarrow$ "quando lo ha visto questo nodo?"
`monotonic()` $\rightarrow$ "quanto tempo è passato localmente?"

Timeout:

$$\text{elapsed} = \text{monotonic}() - \text{start}$$

Log:

2026-05-18 12:30:00.120 node=A event=send

NTP e PTP riducono la distanza tra clock.

- NTP: comune in sistemi generali, accuratezza dipendente da rete e setup.
- PTP: usato quando servono accuratezza maggiore e supporto hardware/rete.
- Slew: correzione graduale del clock.
- Step: salto immediato del clock.

La sincronizzazione fisica riduce l'errore, non lo elimina.

Tempo Come Intervallo

Il modello corretto non è:

$$\text{time} = t$$

È:

$$\text{time} \in [t - \epsilon, t + \epsilon]$$

ϵ rappresenta l'incertezza residua.

Due eventi sono sicuramente ordinabili solo se gli intervalli non si sovrappongono.

Esempio: Intervalli Non Sovrapposti

$e1$ in $[100, 110]$

$e2$ in $[130, 140]$

Possiamo affermare:

$e1$ prima di $e2$

Il margine tra gli intervalli è maggiore dell'incertezza.

Esempio: Intervalli Sovrapposti

e1 in [100, 130]

e2 in [120, 150]

Gli intervalli si sovrappongono.

Non possiamo dimostrare ordine reale usando solo quei timestamp fisici.

Serve un altro contratto: causalità, ordine totale applicativo o consenso.

Timeout

- misura durata locale;
- va implementato con clock monotono;
- non dimostra che un nodo remoto sia morto.

Lease

- diritto valido fino a una scadenza;
- coinvolge clock di più nodi;
- richiede un bound sull'incertezza e margini di sicurezza.

Per progettare un lease bisogna dichiarare:

- errore massimo di sincronizzazione;
- durata del lease;
- bound operativo sulla latenza, se usato;
- margine di sicurezza;
- comportamento quando il clock non è affidabile.

`lease_duration >> clock_uncertainty + network_delay_bound`

Senza assunzioni sostenibili, un lease non è un contratto sicuro.

Lamport definisce una relazione causale:

$$a \rightarrow b$$

Regole:

- ① ordine locale nello stesso processo;
- ② invio di un messaggio prima della ricezione;
- ③ transitività.

Se non vale $a \rightarrow b$ e non vale $b \rightarrow a$:

$$a \parallel b$$

gli eventi sono concorrenti.

Esempio Happened-Before

A: write x=1

A: send m to B

B: receive m

B: read x

Relazioni:

- write x=1 -> send m;
- send m -> receive m;
- receive m -> read x.

Per transitività:

write x=1 -> read x

Esempio: Eventi Concorrenti

A: SET x 1

B: SET x 2

Se non esiste messaggio tra i due eventi:

A.SET || B.SET

Il sistema deve decidere una politica:

- conflitto esplicito;
- last-writer-wins;
- merge applicativo;
- ordine totale imposto.

Un clock di Lamport assegna un intero agli eventi:

$$L : \text{eventi} \rightarrow \mathbb{N}$$

Ogni processo mantiene un contatore locale.

Garanzia:

$$a \rightarrow b \Rightarrow L(a) < L(b)$$

Limite:

$$L(a) < L(b) \not\Rightarrow a \rightarrow b$$

evento locale:

$L_i = L_i + 1$

send:

$L_i = L_i + 1$

allega L_i al messaggio

receive:

$L_i = \max(L_i, \text{timestamp_messaggio}) + 1$

A: local event	$L=1$
A: send m to B	$L=2$
B: local event	$L=1$
B: receive m	$L=\max(1,2)+1=3$
B: local event	$L=4$

Il receive deve avere timestamp maggiore del send.

L'evento locale di B con $L = 1$ non è necessariamente prima dell'evento locale di A con $L = 1$: sono indipendenti.

Lamport: Falso Indizio Di Causalità

A: SET x 1 L=5

B: SET y 1 L=7

5 < 7 non basta per affermare:

A.SET -> B.SET

Lamport preserva causalità nota. Non ricostruisce causalità non osservata.

Lamport: Catena Causale

A: e1 local	L=1
A: send m1 to B	L=2
B: receive m1	L=3
B: send m2 to C	L=4
C: receive m2	L=5

La catena:

A.e1 -> B.receive -> C.receive

rispetta timestamp crescenti lungo tutte le dipendenze note.

Ordine Totale Con Lamport

Per alcuni algoritmi serve ordinare anche eventi concorrenti.

Si usa una coppia:

$$(lamport_time, process_id)$$

Esempio:

$$(4, A) < (4, B)$$

se $A < B$.

È un ordine deterministico, ma può essere artificiale.

Esempio: Mutua Esclusione

Tre processi chiedono la stessa risorsa:

B: request kv:x at L=1

A: request kv:x at L=1

C: request kv:x at L=1

Con tie-breaker $A < B < C$:

$$(1,A) < (1,B) < (1,C)$$

Safety: due processi non entrano insieme.

Liveness: ogni richiesta progredisce se messaggi e nodi corretti restano vivi.

Prima fissiamo una membership ordinata:

$$P = [A, B, C]$$

Un vector clock assegna a ogni evento:

$$V(e) = [clock_A, clock_B, clock_C]$$

Ogni componente ha un proprietario:

- `clock_A`: eventi prodotti da A;
- `clock_B`: eventi prodotti da B;
- `clock_C`: eventi prodotti da C.

Con $P = [A, B, C]$:

- evento locale su A: A incrementa `clock_A`;
- send da A: A incrementa `clock_A` e allega il vettore;
- receive su B: B fa il massimo componente per componente;
- poi B incrementa `clock_B`, perché receive è un evento di B.

Gli altri processi non incrementano la componente di A. Possono solo apprendere un valore più aggiornato tramite merge.

Vector Clock: Esempio Di Merge

$P = [A, B, C]$

A local event: $[1,0,0]$

A sends m to B: $[2,0,0]$

B before receive: $[0,1,0]$

B merges: $\max([0,1,0], [2,0,0]) = [2,1,0]$

B receive event: $[2,2,0]$

Il vettore finale di B dice cosa B conosce degli eventi dei tre processi.

$$a = [2, 0, 0]$$

$$b = [2, 2, 0]$$

$a \leq b$ componente per componente e almeno una componente cresce.

Quindi:

$$a \rightarrow b$$

Il vettore di b include la conoscenza di a .

$$a = [1, 0, 0]$$

$$b = [0, 1, 0]$$

Nessuno dei due vettori domina l'altro.

Quindi:

$$a \parallel b$$

Questa informazione viene persa se imponiamo subito un ordine totale.

Causal delivery è un layer tra trasporto e applicazione:

transport -> causal delivery -> application

Il layer distingue:

receive(m)

deliver(m)

Ricevere un messaggio dalla rete non significa renderlo subito visibile all'applicazione.

Un messaggio non deve essere consegnato prima delle sue cause.

Scenario:

- m1: A pubblica x;
- m2: B reagisce a x;
- C riceve m2 prima di m1.

C deve bufferizzare m2.

Altrimenti l'applicazione vedrebbe la reazione prima della causa.

Causal Delivery: Due Metadati

Un vector clock generale conta eventi:

- computazioni locali;
- aggiornamenti di stato;
- send;
- receive.

Il predicato semplice di causal delivery usa invece un *message vector*:

$$MC(m)[k] = \text{messaggi di } k \text{ noti a } m$$

Se usiamo un vector clock completo, serve anche un sequence number dei messaggi.

Causal Delivery: Stato Locale

Ogni nodo ricevente mantiene un vettore locale:

$P = [A, B, C]$

$\text{delivered} = [\text{delivered}[A], \text{delivered}[B], \text{delivered}[C]]$

$\text{delivered}[k]$ significa:

messaggi inviati da k già consegnati da questo nodo all'applicazione

Non è uno stato globale. Non conta messaggi solo ricevuti dalla rete.

Dal punto di vista del nodo C:

`delivered[A] = 1`

`delivered[B] = 0`

`delivered[C] = 0`

Significa:

- C ha già consegnato un messaggio inviato da A;
- C non ha ancora consegnato messaggi inviati da B;
- i messaggi ricevuti ma bufferizzati non incrementano il contatore.

Un messaggio m inviato dal processo s è consegnabile se:

```
MC(m)[s] = delivered[s] + 1  
for every  $k \neq s$ :  
    MC(m)[k] <= delivered[k]
```

La prima condizione dice: m è il prossimo messaggio atteso da s .

La seconda dice: tutte le cause note prodotte dagli altri processi sono già state consegnate localmente.

Meccanismo	Domanda
Wall-clock locale	Che ora legge questo nodo?
Clock monotono	Quanto tempo è passato localmente?
Clock sincronizzato	In quale intervallo reale?
Lamport clock	Quale ordine rispetta causalità nota?
Ordine totale	Quale ordine deterministico imponiamo?
Vector clock	Gli eventi sono causali o concorrenti?
Causal delivery	Posso consegnare il messaggio ora?

- log operativi: wall-clock;
- retry e timeout: monotonic clock;
- lease: clock sincronizzato con incertezza dichiarata;
- ordinamento di richieste: Lamport più tie-breaker;
- conflitti tra update concorrenti: vector clock;
- replica causale: causal delivery prima di applicare update.

Domanda finale:

quale proprietà temporale promette ogni interfaccia?